Project report

on

Detecting Machine-Generated Code using CodeBERT with Cyclomatic Complexity Analysis

(SemEval-2026 Task 13, Subtask A)

Course code: [CSE---]

Submitted to

Amity University Uttar Pradesh

In partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

In

Computer Science and Engineering

by

Name: Tanush Virmani

Enrollment number: [Enrollment Number]

Under the guidance of

[Faculty Name]

[Designation], [Department]

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AMITY SCHOOL OF ENGINEERING & TECHNOLOGY

AMITY UNIVERSITY UTTAR PRADESH

BATCH: 2023-27

1. INTRODUCTION

Large language models can now generate source code that is fluent, compilable and stylistically similar to code written by people. This makes it increasingly difficult to decide whether a submitted program was authored by a student, a professional developer, or a model such as ChatGPT, GitHub Copilot or Code Llama. The distinction matters for academic integrity, hiring evaluations, open-source provenance and software security. SemEval-2026 Task 13 addresses this problem by asking systems to detect machine-generated code across multiple programming languages, generator families and application scenarios.

This project implements a web-based detector for Subtask A: binary classification of a code snippet as fully human-written or fully machine-generated. The system uses microsoft/codebert-base, a bimodal transformer pretrained on natural language and source code, with a two-class sequence classification head. In addition to authorship prediction, the application analyses cyclomatic complexity of the submitted snippet and plots a per-function complexity graph so that a reviewer can inspect both authorship confidence and structural difficulty in one place.

Key Features of the Proposed System

- CodeBERT-based binary classifier (human-written vs machine-generated).
- Support for the Task 13 training languages: Python, C++ and Java.
- Softmax confidence scores displayed as percentages and as a Gradio label chart.
- Optional language prefix so the model can condition on the selected language.
- Cyclomatic complexity analysis with lizard, covering functions, LOC and average CC.
- Colour-coded horizontal bar chart with Low (≤5), Moderate (6–10) and High (>10) thresholds.
- Gradio web interface with dropdown, code text box, examples and a localhost server.
- Checkpoint loading from ./checkpoints or MODEL_CHECKPOINT; fallback to base CodeBERT.

Project Workflow

The user selects a programming language, pastes a snippet and clicks Analyse. The snippet is tokenised (maximum 512 tokens), passed through CodeBERT, and converted to class probabilities with softmax. In parallel, lizard parses the same text and computes cyclomatic complexity for each detected function. The interface then shows the predicted label, human and machine confidence, a complexity summary table and the complexity graph.

Figure 1. Code-to-prediction workflow: language selection, CodeBERT classification, complexity analysis and visualisation.

2. PROBLEM STATEMENT AND OBJECTIVES

A single binary label is not enough for a teacher or reviewer. They also need to know how confident the model is, which language was assumed, and whether the submitted program is structurally simple or highly branched. Existing detectors are often offline scripts, language-specific, or black-box APIs. This project therefore builds an interactive detector that is aligned with SemEval-2026 Task 13

Subtask A and adds an independent complexity module.

2.1 Problem Statement

Given a source-code snippet in Python, C++ or Java, the system must predict whether the snippet is fully human-written (label 0) or fully machine-generated (label 1), report calibrated-style class probabilities, and estimate cyclomatic complexity of the functions it contains. The detector should run locally in a browser, accept pasted code, and remain usable even when a fine-tuned checkpoint is not yet available by falling back to microsoft/codebert-base.

2.2 Objectives

- To implement a CodeBERT sequence classifier for human vs machine-generated code.
- To expose language selection, code input, prediction and confidence in a Gradio UI.
- To compute cyclomatic complexity, LOC and per-function ratings for Python, C++ and Java.
- To plot a complexity graph with explicit Low / Moderate / High thresholds.
- To support fine-tuned checkpoints without changing the inference interface.
- To document the design so that the experiment can be reproduced on localhost.

2.3 Scope

The present implementation is a research prototype for academic demonstration of Subtask A. It does not attempt Subtask B (which LLM wrote the code) or Subtask C (hybrid / adversarial authorship). Complexity analysis is static and language-aware via lizard; it is not a runtime profiler and does not prove Big-O complexity. Competitive leaderboard performance requires fine-tuning on the official Task 13 dataset. Until a checkpoint is saved under ./checkpoints, the classification head of the public CodeBERT checkpoint is used as a runnable baseline.

3. PROPOSED SYSTEM DESIGN AND ARCHITECTURE

The system follows a layered design. The input layer captures language and source text. The encoding layer tokenises the snippet for CodeBERT. The decision layer produces class logits and softmax probabilities. A parallel analysis layer computes structural metrics and a matplotlib figure. The presentation layer is a Gradio Blocks application served at 127.0.0.1:7860.

Figure 2. Proposed system architecture: Gradio UI, CodeBERT classifier and lizard complexity analyser.

3.1 Major Modules

Module	Purpose
Language selector	Dropdown for Python, C++ and Java; used for language prefix and lizard file extension.
Code input	Multi-line text box for the snippet, with two Gradio examples.
Tokenizer	AutoTokenizer from microsoft/codebert-base; truncation and padding to 512 tokens.
Classifier	AutoModelForSequenceClassification, 2 labels: Human-written and Machine-generated.
Confidence display	Softmax probabilities shown as Label bars and as percentage text boxes.

Module	Purpose
Complexity engine	lizard.analyze_file.analyze_source_code for CC, nloc, tokens and function list.
Complexity plot	Horizontal bar chart coloured by rating; dashed lines at CC = 5 and CC = 10.

Table 1. Major system modules.

3.2 System Data Flow

On Analyse, run_inference calls predict and analyze_complexity with the same (language, code) pair. predict prepends a language comment, tokenises the text, moves tensors to CUDA if available otherwise CPU, and applies softmax to logits. The predicted class is the argmax of the two probabilities. analyze_complexity maps the language to snippet.py, snippet.cpp or snippet.java, parses functions, rates the maximum cyclomatic complexity, and builds a figure only when at least one function is detected. Empty input returns an error message without loading the transformer a second time beyond the cached first load.

4. DATA SOURCES AND PREPROCESSING

SemEval-2026 Task 13 provides official training data covering C++, Python and Java in an algorithmic (for example LeetCode-style) domain. Evaluation also considers unseen languages (Go, PHP, C#, C, JavaScript) and unseen domains (research and production GitHub code). The Hugging Face dataset DaniilOr/SemEval-2026-Task13 is the intended training source. Task rules disallow extra training data and models pretrained specifically for AI-generated code detection; general-purpose code models such as CodeBERT are permitted.

4.1 Dataset Details

Parameter	Training (Subtask A)	Notes
Languages	C++, Python, Java	Unseen languages appear only at test time.
Domain	Algorithmic	Research / production are OOD evaluation settings.
Labels	0 = human, 1 = machine	Binary Subtask A setting.
Metric	Macro-averaged F1	Primary official metric.
Generator metadata	Must not be used	Task restriction.
Demo examples	Two Python snippets	Human-style two-sum; verbose Fibonacci.

Table 2. Task 13 data characteristics used to design the prototype.

4.2 Input Encoding

RoBERTa/CodeBERT tokenisation is applied with truncation at 512 tokens, which is the standard maximum sequence length of codebert-base. Longer programs therefore lose tokens from the end. A short language hint is prepended (# language: python or // language: cpp / java) so that a fine-tuned model can use language as a condition if that convention is used during training.

4.3 Complexity Parsing

Lizard is a language-aware static analyser. It counts independent paths through each function (McCabe cyclomatic complexity) together with non-comment lines of code. If no function is found, the UI still reports file-level LOC and token count and asks the user to paste a complete function or class. Parse failures are caught and shown as an error rather than crashing the web app.

5. MACHINE LEARNING MODEL AND METHODOLOGY

5.1 CodeBERT Encoder

CodeBERT (Feng et al., 2020) is a bimodal Transformer pretrained with masked language modelling and replaced token detection on paired natural language and code from CodeSearchNet. The base checkpoint has 12 layers, hidden size 768 and a RoBERTa-style tokenizer. For this project it is loaded as AutoModelForSequenceClassification with num_labels = 2, id2label mapping 0 to Human-written and 1 to Machine-generated.

5.2 Inference Procedure

The model is set to eval() and weights are cached with functools.lru_cache so the checkpoint is loaded once per process. Inference uses torch.no_grad(). Logits are converted with softmax. The predicted class is human if P(human) ≥ P(machine), otherwise machine. Confidence reported as “top confidence” is the maximum of the two class probabilities.

5.3 Fine-tuning Path

For a competitive SemEval submission the classification head and encoder should be fine-tuned only on the official Subtask A training set. Recommended practice from participating teams includes stratified balancing, threshold calibration and caution against aggressive unfreezing, which can harm out-of-distribution generalisation. After training, save Hugging Face weights to ./checkpoints/. The app prefers that directory when it exists; otherwise it loads microsoft/codebert-base. Environment variables MODEL_NAME, MODEL_CHECKPOINT and MAX_LENGTH override defaults.

5.4 Complexity Ratings

Cyclomatic complexity	Rating	Interpretation
1–5	Low	Few branches; generally easy to test.
6–10	Moderate	Noticeable branching; still maintainable.
11 and above	High	Many independent paths; harder to test.

Table 3. Cyclomatic complexity rating scheme used in the UI.

6. VISUALISATION AND EVIDENCE GENERATION

Rather than returning only a class name, the interface presents several pieces of evidence that a human reviewer can inspect together.

6.1 Authorship Confidence

gr.Label renders a two-class probability bar. Separate read-only text boxes show human-written and machine-generated confidence as percentages. A markdown panel repeats language, predicted label

and top confidence, and cites SemEval-2026 Task 13 Subtask A.

6.2 Complexity Table

The summary reports overall rating (from maximum CC), average cyclomatic complexity, lines of code and number of functions. A markdown table lists each function name, CC, rating and LOC.

6.3 Complexity Graph

A horizontal bar chart plots one bar per function. Green, orange and red encode Low, Moderate and High. Dashed vertical lines mark the Moderate threshold (5) and High threshold (10). Numeric CC values are annotated at the end of each bar. Long function names are truncated. The y-axis is inverted so the first detected function appears at the top. Matplotlib uses the Agg backend so the figure can be generated on a headless localhost server.

Figure 3. Example complexity graph: cyclomatic complexity by function with Low and High threshold lines.

6.4 Responsible Interpretation

CodeBERT probabilities are not proof of cheating. Unfine-tuned heads can be poorly calibrated. Cyclomatic complexity measures control-flow structure, not whether an LLM wrote the code. Both outputs should assist a human reviewer.

7. IMPLEMENTATION AND ENVIRONMENT

7.1 Software and Hardware

Component	Specification
Language	Python 3.14 (virtual environment .venv)
Web UI	Gradio 6.x, Blocks API, theme Soft, ssr_mode=False
ML stack	PyTorch 2.x, Hugging Face Transformers
Complexity	lizard ≥ 1.17
Plotting	matplotlib ≥ 3.7, Agg backend
Hardware	CPU sufficient for demo; CUDA used automatically if present
Server	http://127.0.0.1:7860

Table 4. Implementation environment.

7.2 Application Structure

The repository is intentionally small. app.py contains constants, model loading, prediction, complexity analysis and the Gradio layout. requirements.txt pins gradio, torch, transformers, lizard and matplotlib. Fine-tuned weights, if any, live in checkpoints/. The model is loaded lazily on first Analyse click so the UI can appear immediately.

7.3 Running the Prototype

- Create and activate a virtual environment.

- Install dependencies with pip install -r requirements.txt.
- Run python app.py and open http://127.0.0.1:7860.
- Select a language, paste code or click an example, then Analyse code.
- Optional: export MODEL_CHECKPOINT=./checkpoints after fine-tuning.

8. RESULTS AND PERFORMANCE ANALYSIS

This section reports what the implemented prototype currently demonstrates, and what published Task 13 systems show is achievable after proper fine-tuning. Invented leaderboard scores for this repository are not claimed.

8.1 Functional Results of the Prototype

Capability	Observed behaviour
Local web app	Gradio serves the UI at 127.0.0.1:7860 after disabling SSR.
Example 1 (two_sum)	Lizard typically reports Low complexity (CC around 3) for the hash-map loop.
Example 2 (Fibonacci)	Simple loop plus guards remains Low; verbose docstring does not raise CC.
Empty input	Prediction shows an error; complexity asks the user to paste code.
No functions	LOC and token count still appear; plot is omitted.
Base CodeBERT head	Runnable without a checkpoint; class scores are not yet task-calibrated.

Table 5. Functional results of the current application.

8.2 Expected Detection Performance after Fine-tuning

Participating SemEval-2026 Task 13 teams that fine-tuned CodeBERT report a consistent pattern: in-domain (seen language, algorithmic) scores can be very high, while out-of-distribution language and domain shifts are much harder. Team Duo reported near-perfect in-distribution F1 (about 0.99) that dropped to about 0.35 under OOD evaluation, with over-prediction of the machine class. Team Omega found that a frozen encoder was more stable OOD (macro F1 about 0.51) than a partially fine-tuned model that collapsed to about 0.30 OOD despite 0.98 in-domain F1. Other CodeBERT systems reported private-leaderboard macro F1 in roughly the 0.53–0.56 range after calibration. These figures are literature context, not measurements of the unfine-tuned demo.

Setting	Typical CodeBERT pattern	Implication for this project
Seen languages, algorithmic	High accuracy / F1 after fine-tuning	Fine-tune on official train split.
Unseen languages	Sharp drop; syntax shift	Do not overfit Python/C++/Java style.
Unseen domains	Research / production code harder	Calibrate threshold; avoid overconfident machine labels.

Table 6. Qualitative performance picture from Task 13 CodeBERT papers.

8.3 Interpretation

The prototype succeeds as an end-to-end tool: a reviewer can paste code, see authorship probabilities and inspect complexity. Detection quality will track checkpoint quality. Complexity scores are deterministic given the parser and are therefore a reliable second channel even when the neural classifier is still a baseline.

9. ETHICAL CONSIDERATIONS

• Academic integrity: a high machine-generated score must not automatically become a plagiarism verdict. Students may use assistants for comments, renaming or small edits. The tool is for assistance, not automatic punishment.

• False positives: human code that is verbose, templated or copied from tutorials can resemble LLM style. False accusations can harm students and employees.

• False negatives: lightly edited LLM code, hybrid authorship and adversarial prompting are out of scope of Subtask A and can evade a binary detector.

• Bias: CodeBERT and Task 13 training languages emphasise Python, C++ and Java. Performance may degrade on other languages and coding cultures.

• Privacy: pasted code may contain secrets, student identifiers or proprietary logic. The localhost deployment keeps data on the user's machine; a public share link should not be enabled for confidential coursework.

• Transparency: confidence and complexity charts are explanations of model and parser outputs, not legal proof of authorship.

• Human oversight: instructors should inspect the original snippet, the language setting and the complexity graph before any disciplinary action.

10. TESTING AND VALIDATION

10.1 Validation Plan

Test	Expected outcome
App launch	Server starts on port 7860 without hanging.
Empty code	Error markdown; no crash.
Python two_sum example	Functions detected; Low CC; plot drawn.
C++ / Java extension mapping	lizard receives .cpp or .java filename.
Long snippet	Tokenizer truncates at 512 tokens.
Checkpoint missing	Loads microsoft/codebert-base.
Checkpoint present	Loads ./checkpoints if it is a directory.
CUDA available	Model moved to GPU; otherwise CPU.

Table 7. Functional validation plan.

10.2 Model Validation (after fine-tuning)

- Use only official Task 13 splits; no extra data.
- Select checkpoints by validation macro F1.
- Report accuracy, per-class precision/recall, confusion matrix and macro F1.
- Evaluate seen-language and, when labels allow, OOD language/domain slices.
- Do not use generator identity as a feature.

10.3 Robustness

The UI must survive incomplete functions, syntax errors and empty boxes. Lizard exceptions are converted to a short error string. Gradio SSR is disabled because it can stall startup when Node is unavailable. Matplotlib Agg avoids macOS GUI backend deadlocks.

11. ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

11.1 Advantages

- Interactive localhost demo rather than a command-line-only classifier.
- Joint view of neural authorship scores and classical complexity metrics.
- Language-aware parsing for the three official training languages.
- Checkpoint hook that does not require rewriting the interface after training.
- Clear visual thresholds that non-ML reviewers can understand.

11.2 Limitations

- Without fine-tuning, CodeBERT's randomly initialised classification head is not a reliable detector.
- Truncation at 512 tokens discards the tail of long files.
- Subtasks B and C are not implemented.
- Complexity is McCabe CC, not asymptotic time or space complexity.
- OOD languages in the official test sets are outside the UI dropdown.
- No user study of whether complexity charts improve reviewer decisions.

11.3 Future Scope

- Fine-tune CodeBERT on DaniilOr/SemEval-2026-Task13 and calibrate the decision threshold.
- Add chunked inference with aggregation for files longer than 512 tokens.
- Extend language support toward Go, PHP, C#, C and JavaScript.
- Compare UniXcoder, GraphCodeBERT or CodeT5+ under the same UI.
- Add Subtask B (generator attribution) and Subtask C (hybrid authorship).
- Export a PDF report of prediction, confidence and the complexity figure.

12. CONCLUSION AND REPRODUCIBILITY

12.1 Overall Results

The project delivers a complete local web application for SemEval-2026 Task 13 Subtask A. It classifies pasted source code with CodeBERT, reports class confidence, and analyses cyclomatic complexity with a colour-coded graph. The architecture is aligned with published Task 13 systems that use CodeBERT as the encoder, while the Gradio interface makes the method usable by a non-specialist. Detection accuracy after fine-tuning is expected to be strong in-domain and weaker under language and domain shift, which is the central scientific challenge of the task.

- A runnable Gradio app at http://127.0.0.1:7860 was implemented in app.py.
- microsoft/codebert-base is used as the default encoder with a two-class head.
- Lizard provides per-function cyclomatic complexity, LOC and a matplotlib plot.
- Two sample Python examples demonstrate typical Low-complexity algorithmic snippets.
- Fine-tuned weights can be dropped into ./checkpoints without changing the UI contract.

12.2 Conclusion

Machine-generated code detection is both a modelling problem and a human-review problem. This project treats it as such: CodeBERT supplies a learned authorship signal, complexity analysis supplies an independent structural signal, and the browser UI keeps a person in the loop. The next required step for competitive evaluation is supervised fine-tuning on the official dataset, with careful attention to out-of-distribution generalisation.

12.3 Reproducibility Checklist

Artifact	Purpose
app.py	Model loading, inference, complexity, Gradio layout
requirements.txt	gradio, torch, transformers, lizard, matplotlib
checkpoints/	Optional fine-tuned Hugging Face weights
microsoft/codebert-base	Public encoder used when no checkpoint exists
DaniilOr/SemEval-2026-Task13	Official data for training and evaluation
Project_Report.pdf	This report

Table 8. Key project artifacts for reproducibility.

13. REFERENCES

[1] Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., & Zhou, M. (2020). CodeBERT: A pre-trained model for programming and natural languages. Findings of EMNLP 2020, 1536–1547.

[2] Orel, D., et al. (2026). SemEval-2026 Task 13: Detecting machine-generated code with multiple programming languages, generators, and application scenarios. Proceedings of SemEval 2026.

[3] G, S., M, S., Sivanaiah, R., & S, A. D. (2026). Team Duo at SemEval-2026 Task 13: Fine-tuning CodeBERT for out-of-distribution AI-generated code detection. SemEval 2026, 2104–2107.

[4] Team_Omega. (2026). Frozen vs. trainable representations for out-of-distribution AI-generated code detection: A CodeBERT fine-tuning study. SemEval 2026.

[5] McCabe, T. J. (1976). A complexity measure. IEEE Transactions on Software Engineering, SE-2(4), 308–320.

[6] Yin, T. Lizard: A simple code complexity analyser. https://github.com/terryyin/lizard

[7] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). RoBERTa: A robustly optimized BERT pretraining approach. arXiv:1907.11692.

[8] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, ■., & Polosukhin, I. (2017). Attention is all you need. NeurIPS 2017, 30.

[9] Wolf, T., et al. (2020). Transformers: State-of-the-art natural language processing. EMNLP 2020 System Demonstrations, 38–45.

[10] Abid, A., Abdalla, A., Abid, A., Khan, D., Alfozan, A., & Zou, J. (2019). Gradio: Hassle-free sharing and testing of ML models in the wild. arXiv:1906.02569.

[11] Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. NeurIPS 2019, 32.

[12] Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., Tufano, M., Deng, S. K., Clement, C., Drain, D., Sundaresan, N., Yin, J., Jiang, D., & Zhou, M. (2021). GraphCodeBERT: Pre-training code representations with data flow. ICLR 2021.

[13] Husain, H., Wu, H.-H., Gazit, T., Allamanis, M., & Brockschmidt, M. (2019). CodeSearchNet challenge: Evaluating the state of semantic code search. arXiv:1909.09436.

[14] SemEval-2026 Task 13 dataset. Hugging Face: DaniilOr/SemEval-2026-Task13. GitHub: mbzuai-nlp/SemEval-2026-Task13.